SRI ESHWAR COLLEGE OF ENGINEERING

Department of CSE (Artificial Intelligence and Machine Learning)

CLOUD COMPUTING IMPLEMENTATION REPORT

Docker • GitHub Actions • Amazon ECR • Amazon ECS Fargate

Name	Kanimozhi R
Department / Section	CSE (AIML) – A
Subject	Cloud Computing Technologies
Course Code	U23IT4AS4 / U23IT402 – Cloud Computing
Project Title	Intern application-Cloud Deployment
Deployment Approach	Docker → GitHub Actions → Amazon ECR → Amazon ECS Fargate

1. Objective

To design and implement the AWS infrastructure required to host the Ticket Booking Application by containerizing the application using Docker, automating the build and image publishing process with GitHub Actions, storing Docker images in Amazon ECR, and deploying the application using Amazon ECS with the Fargate launch type.

2. Technologies and AWS Services Used

Component	Technology / Service
Application	Intern Application
Frontend	React + Vite
Backend	Node.js + Express
Database	MongoDB Atlas
Containerization	Docker and Docker Compose
CI/CD	GitHub Actions
Container Registry	Amazon ECR
Cloud Deployment	Amazon ECS Fargate
AWS Region	Asia Pacific (Mumbai) – ap-south-1

3. Step-by-Step Implementation

Step 1 – Create Dockerfiles

Dockerfiles were created for the Ticket Booking Application frontend and backend. The frontend uses a multi-stage build with Node.js and Nginx, while the backend runs on Node.js and exposes port 5000.

Step 2 – Configure Docker Compose

Docker Compose was configured to build and run the frontend and backend services together. The frontend is mapped to port 3000 and the backend to port 5000.

Step 3 – Build and Test Docker Images

The Docker images were built locally and the application was tested through Docker Desktop. This verified that the containerized application could start successfully.

Step 4 – Push Source Code to GitHub

The Ticket Booking Application source code and the CI/CD workflow were maintained in a GitHub repository. The workflow file was placed under .github/workflows/ci.yml.

Step 5 – Configure GitHub Actions

GitHub Actions was configured to checkout the source code, install dependencies, build the frontend, install backend dependencies, authenticate with AWS using OIDC, log in to ECR, build Docker images, and push the images to ECR.

Step 6 – Store Images in Amazon ECR

Separate private ECR repositories were used for the frontend and backend Docker images. The workflow successfully published the images using the latest tag.

Step 7 – Deploy Using Amazon ECS Fargate

An ECS cluster was configured for the Ticket Booking Application. Separate frontend and backend task definitions and services were deployed using the Fargate launch type.

Step 8 – Verify the Deployed Application

The ECS services were started and the deployed Ticket Booking Application was accessed through its public endpoint. The application interface was verified successfully.

4. Implementation Screenshots

Frontend Dockerfile using Node.js and Nginx

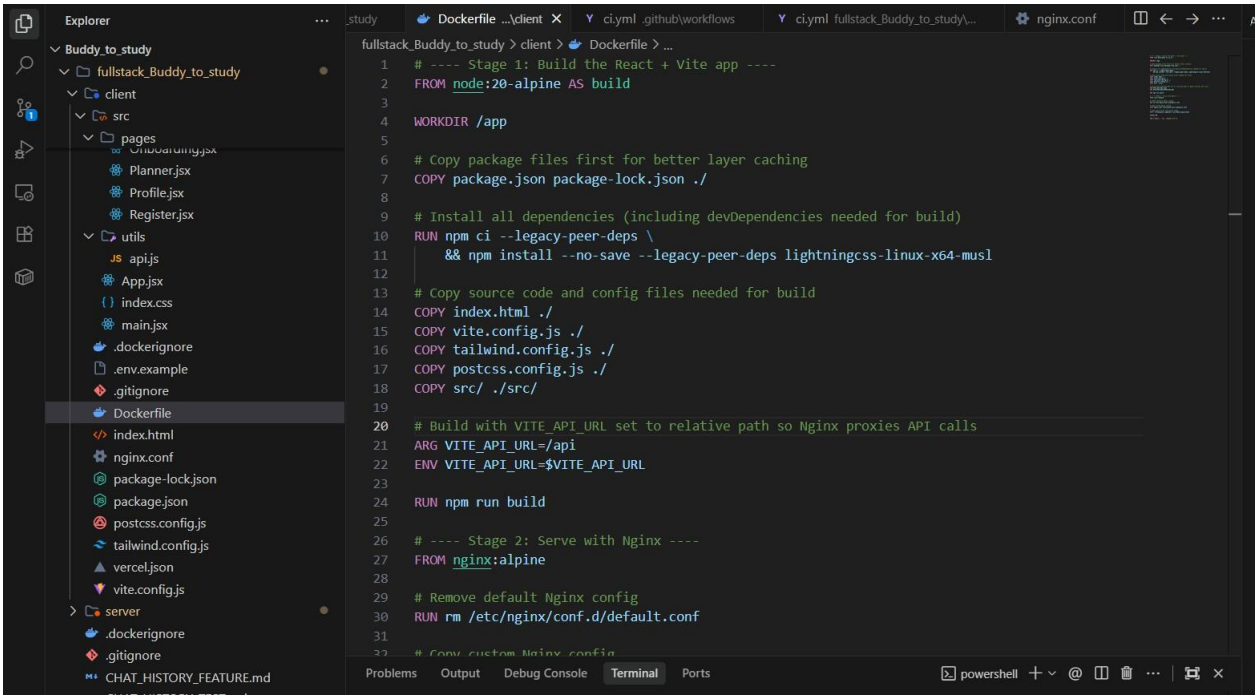


Figure 1. Frontend Dockerfile using Node.js and Nginx

Docker Compose configuration for frontend and backend

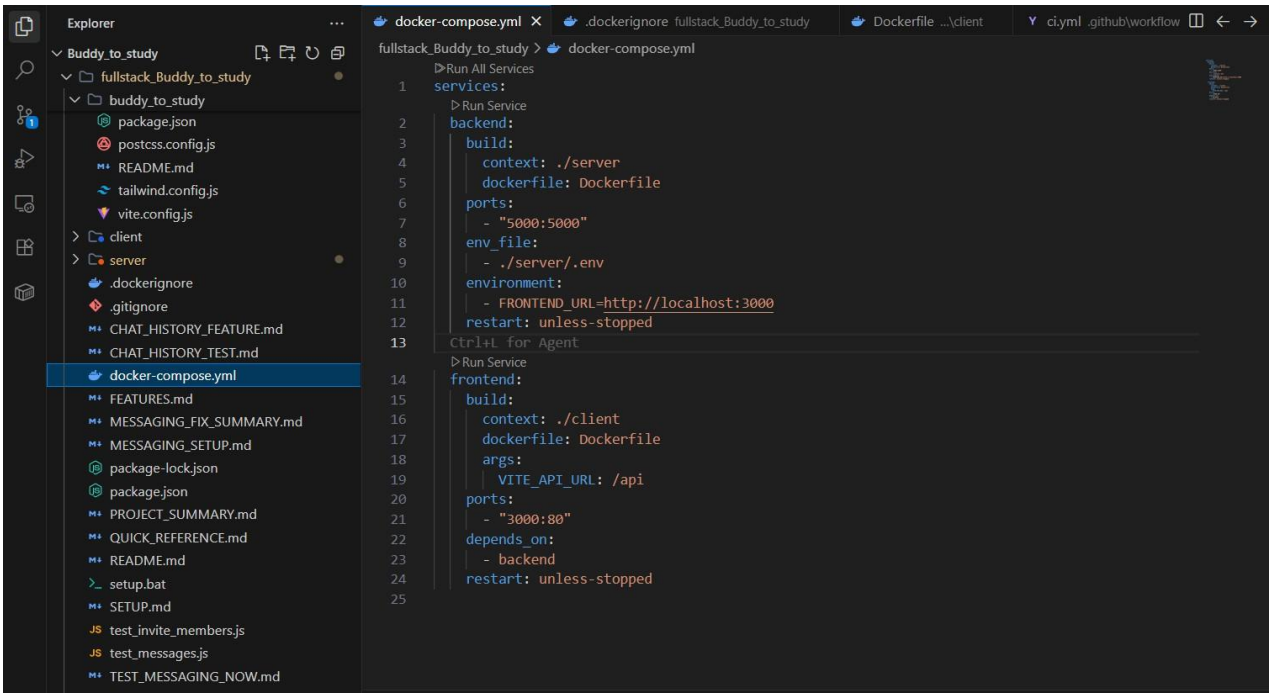


Figure 2. Docker Compose configuration for frontend and backend

Backend Dockerfile exposing port 5000

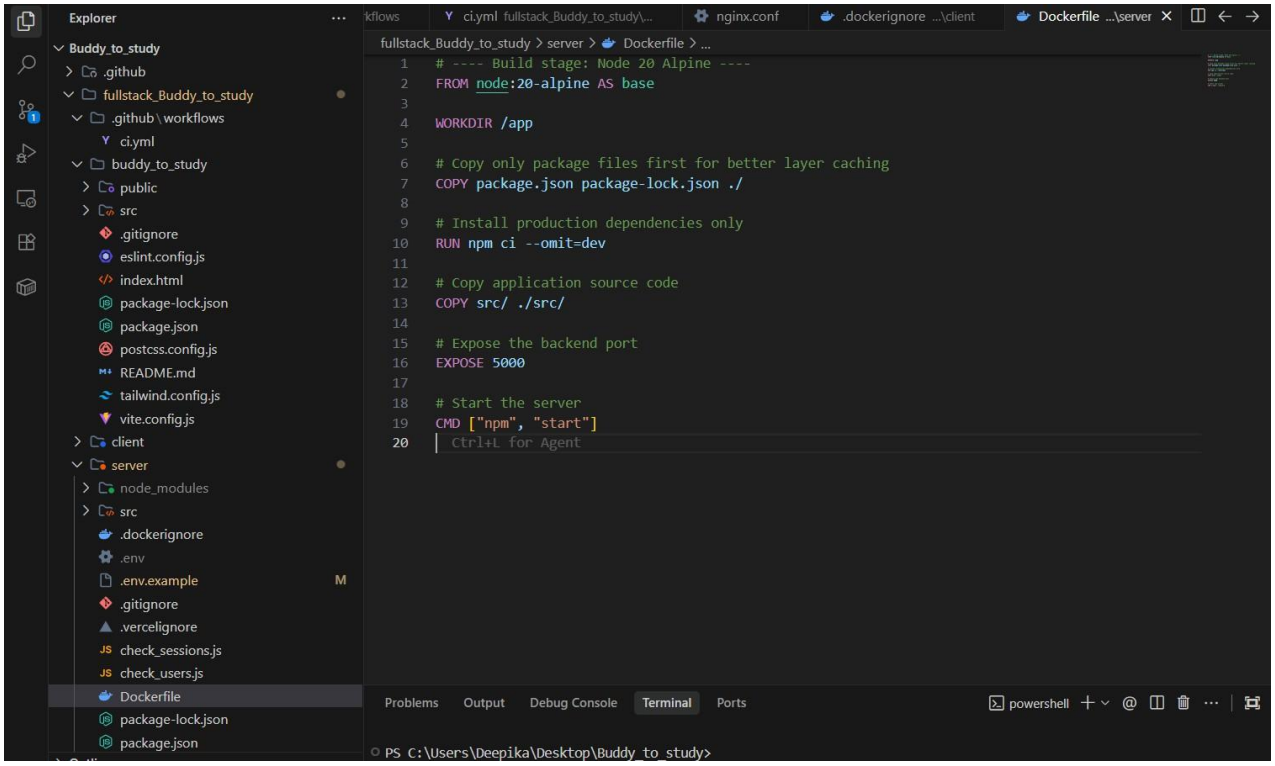


Figure 3. Backend Dockerfile exposing port 5000

Docker Desktop showing frontend and backend images

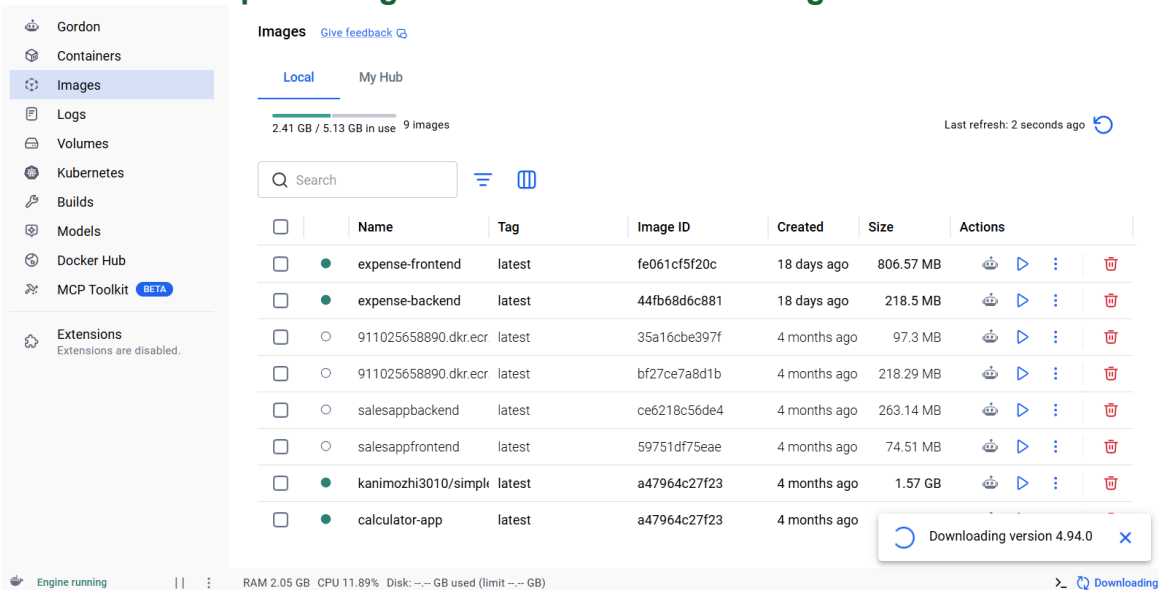


Figure 4. Docker Desktop showing frontend and backend images

Figure 5. Ticket Booking application running locally through Docker

Application source repository on GitHub

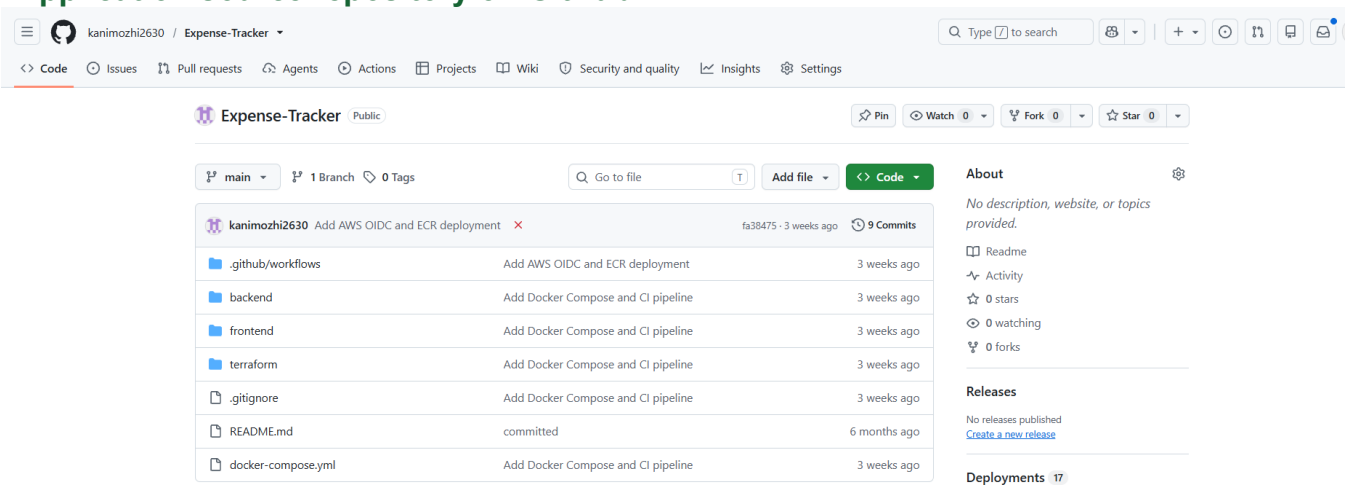


Figure 6. Ticket Booking application source repository on GitHub

Successful GitHub Actions CI/CD workflow

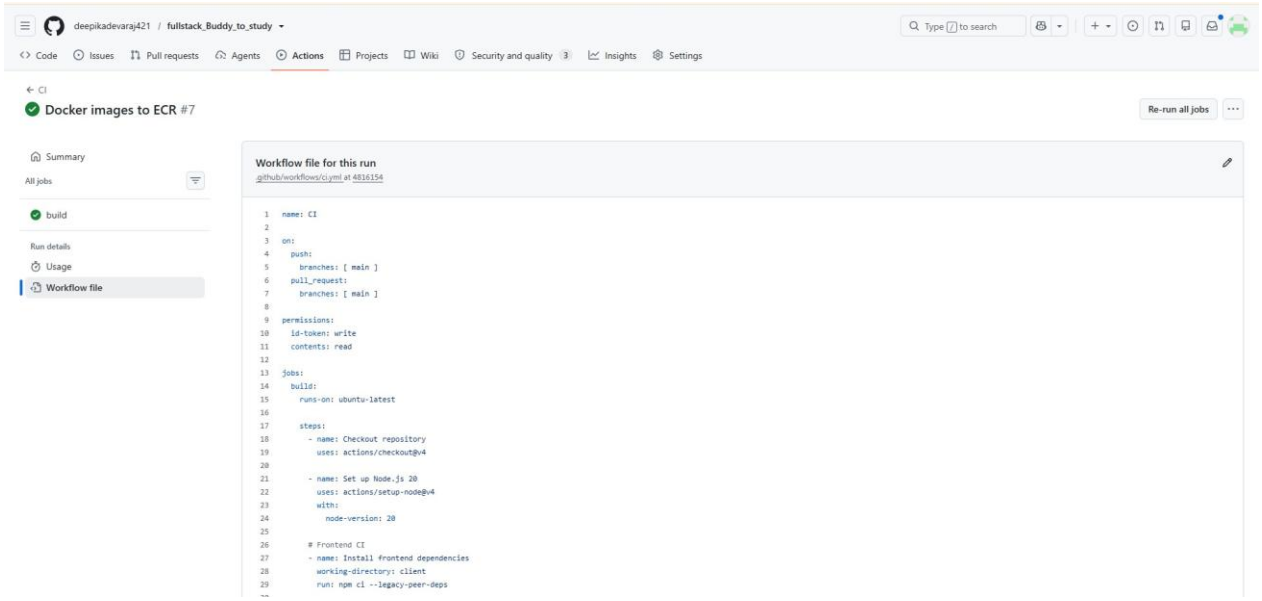


Figure 7. Successful GitHub Actions CI/CD workflow

Amazon ECR repositories for the Ticket Booking application

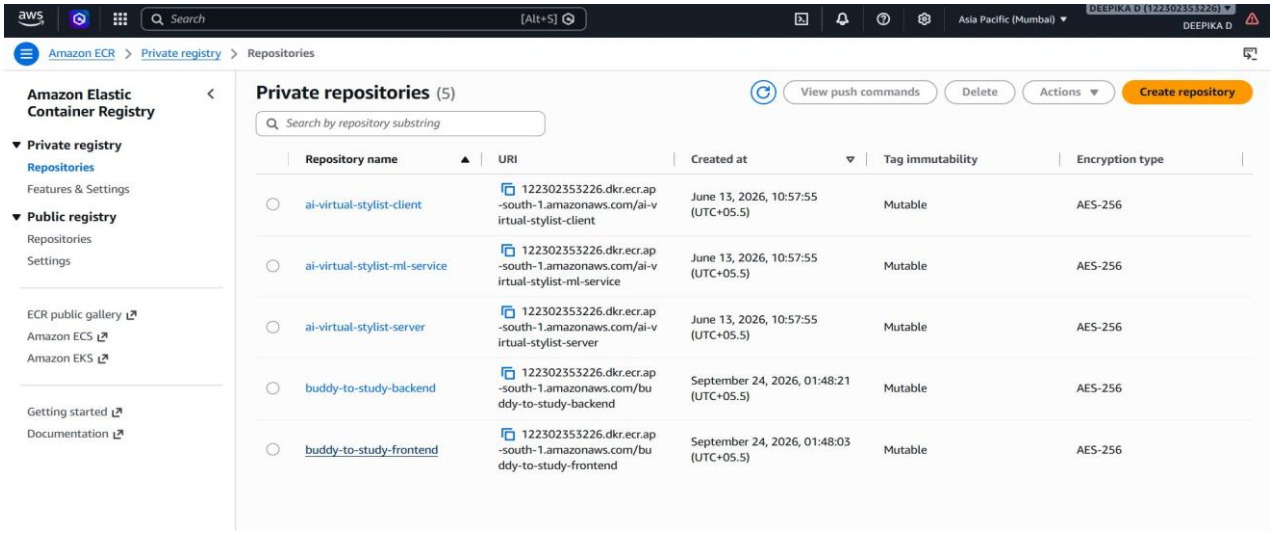


Figure 8. Amazon ECR repositories for the Ticket Booking application

ECS Fargate cluster with frontend and backend services

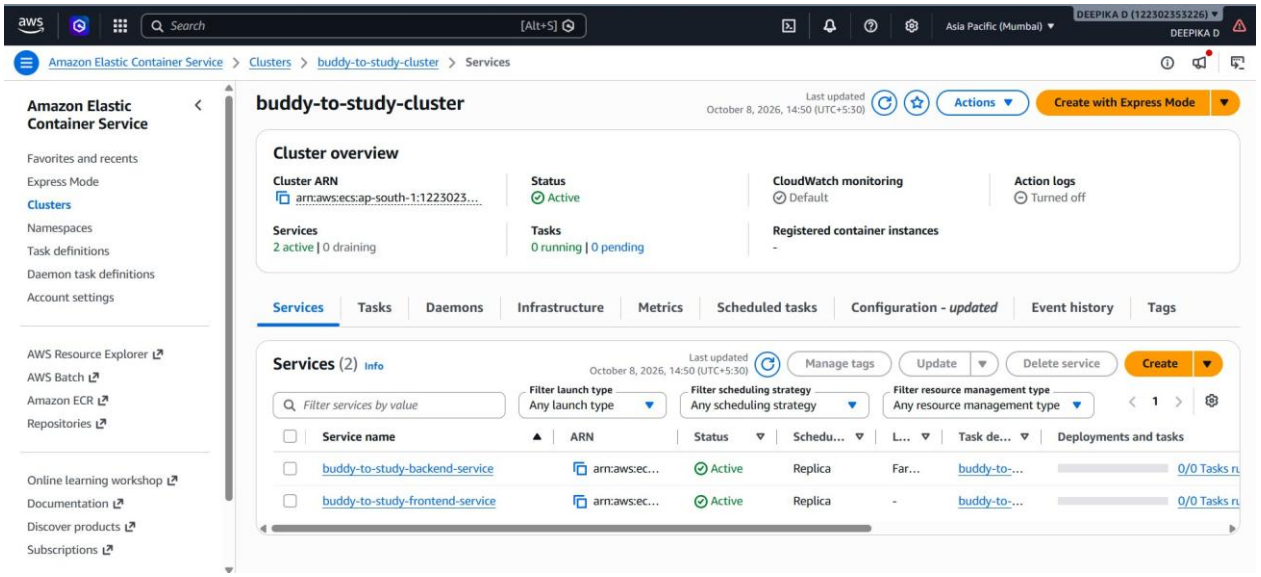


Figure 9. ECS Fargate cluster with frontend and backend services

ECS task definitions for the Ticket Booking application

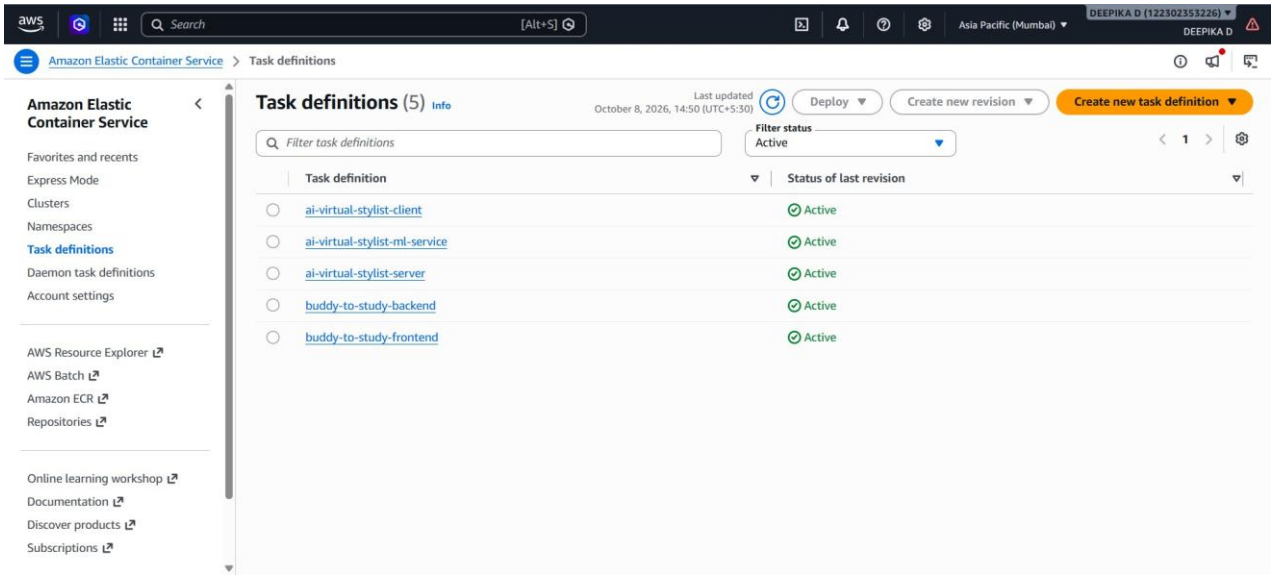


Figure 10. ECS task definitions for the Ticket Booking application

5. Deployment Flow

1 GitHub Repository

↓

2 GitHub Actions – CI/CD

↓

3 Docker Image Build

↓

4 Amazon ECR

- Ticket Booking Frontend Image
- Ticket Booking Backend Image

↓

5 Amazon ECS Fargate

- Frontend Service – Port 80
- Backend Service – Port 5000

↓

6 MongoDB Atlas

6. Result

The Ticket Booking Application was successfully containerized using Docker. The Docker images were built through GitHub Actions and pushed to Amazon ECR. The frontend and backend were deployed as ECS Fargate services, and the application interface was verified through the deployed public endpoint.

7. Conclusion

This implementation demonstrates a practical cloud deployment workflow for the Ticket Booking Application. The workflow connects source control, continuous integration and deployment, Docker containerization, Amazon ECR image storage, and Amazon ECS Fargate hosting into a single deployment pipeline.

8. Note

The screenshots included in this report document the Docker, GitHub Actions, ECR, ECS Fargate, and application verification stages of the implementation. AWS resources used for demonstration can be stopped or removed after the practical session to avoid unnecessary ongoing usage charges.